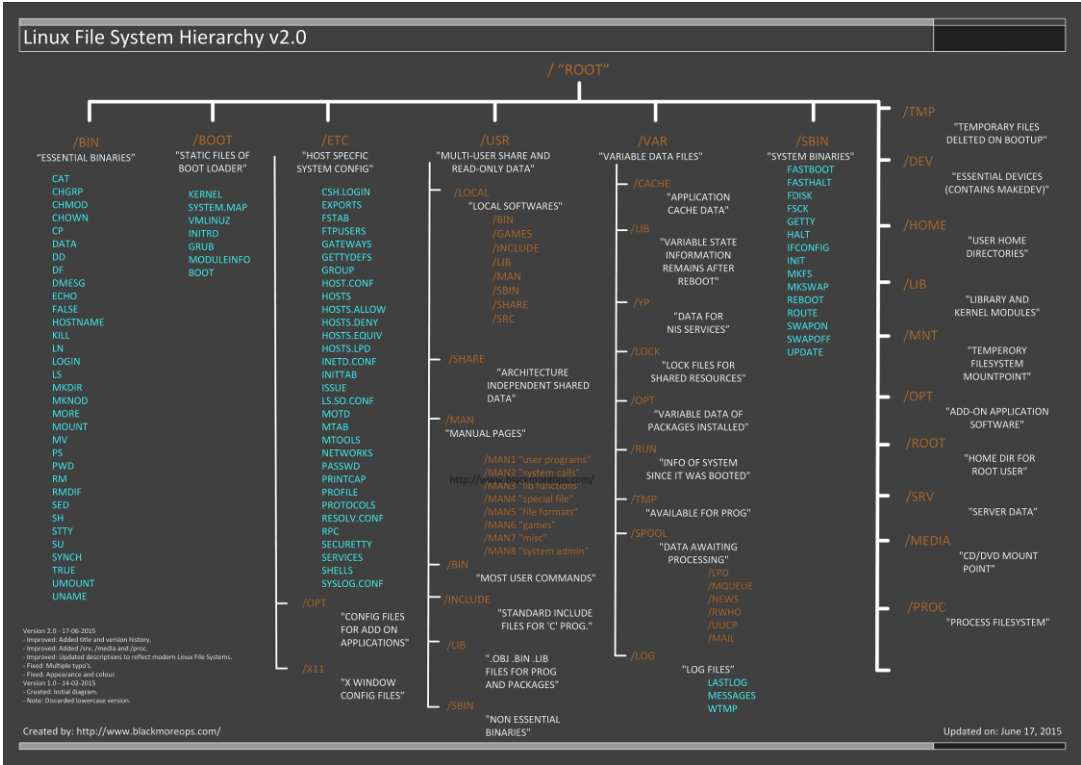


Linux 文件系统结构介绍



Linux 中的文件是什么？它的文件系统又是什么？那些配置文件又在哪里？我下载好的程序保存在哪里了？在 Linux 中文件系统是标准结构的吗？好了，上图简明地阐释了 Linux 的文件系统的层次关系。当你苦于寻找配置文件或者二进制文件的时候，这便显得十分有用了。我在下方添加了一些解释以及例子，不过“篇幅较长，可以有空再看”。

另外一种情况便是当你在系统中获取配置以及二进制文件时，出现了不一致性问题，如果你是在一个大型组织中，或者只是一个终端用户，这也有可能破坏你的系统（比如，二进制文件运行在旧的库文件上了）。若然你在你的 Linux 系统上做安全审计的话，你将会发现它很容易遭到各种攻击。所以，保持一个清洁的操作系统（无论是 Windows 还是 Linux）都显得十分重要。

Linux 的文件是什么？

对于 UNIX 系统来说(同样适用于 Linux)，以下便是对文件简单的描述：

在 UNIX 系统中，一切皆为文件；若非文件，则为进程

这种定义是比较正确的，因为有些特殊的文件不仅仅是普通文件（比如命名管道和套接字），不过为了让事情变的简单，“一切皆为文件”也是一个可以让人接受的说法。Linux 系统也像 UNIX 系统一样，将文件和目录视如同物，因为目录只是一个包含了其他文件名的文件而已。程序、服务、文本、图片等等，都是文件。对于系统来说，输入和输出设备，基本上所有的设备，都被当做是文件。

题图版本历史：

- Version 2.0 - 17-06-2015
 - - Improved: 添加标题以及版本历史
 - - Improved: 添加/srv, /meida 和/proc
 - - Improved: 更新了反映当前的 Linux 文件系统的描述
 - - Fixed: 多处的打印错误
 - - Fixed: 外观和颜色
- Version 1.0 - 14-02-2015
 - - Created: 基本的图表
 - - Note: 摒弃更低的版本

下载链接

以下是大图的下载地址。如果你需要其他格式，请跟原作者联系，他会尝试制作并且上传到某个地方以供下载

- [大图 \(PNG 格式\) - 2480×1755 px - 184KB](#)
- [最大图 \(PDF 格式\) - 9919x7019 px - 1686KB](#)

注意：PDF 格式文件是打印的最好选择，因为它画质很高。

Linux 文件系统描述

为了有序地管理那些文件，人们习惯把这些文件当做是硬盘上的有序的树状结构，正如我们熟悉的’MS-DOS’ (磁盘操作系统) 就是一个例子。大的分枝包括更多的分枝，分枝的末梢是树的叶子或者普通的文件。现在我们将会以这树形图为例，但晚点我们会发现为什么这不是一个完全准确的一幅图。

目录	描述
	主层次 的根，也是整个文件系统层次结构的根目录

目录	描述
<code>/bin</code>	存放在单用户模式可用的必要命令二进制文件，所有用户都可用，如 <code>cat</code> 、 <code>ls</code> 、 <code>cp</code> 等等
<code>/boot</code>	存放引导加载程序文件，例如 <code>kernels</code> 、 <code>initrd</code> 等
<code>/dev</code>	存放必要的设备文件，例如 <code>/dev/null</code>
<code>/etc</code>	存放主机特定的系统级配置文件。其实这里有个关于它名字本身意义上的争议。在贝尔实验室的 UNIX 实施文档的早期版本中， <code>/etc</code> 表示是“其他 (etcetera) 目录”，因为从历史上看，这个目录是存放各种不属于其他目录的文件（然而，文件系统目录标准 FSH 限定 <code>/etc</code> 用于存放静态配置文件，这里不该存有二进制文件）。早期文档出版后，这个目录名又重新定义成不同的形式。近期的解释中包含着诸如“可编辑文本配置”或者“额外的工具箱”这样的重定义
<code>/etc/opt</code>	存储着新增包的配置文件 <code>/opt/</code> 。
<code>/etc/sgml</code>	存放配置文件，比如 <code>catalogs</code> ，用于那些处理 SGML (译者注：标准通用标记语言) 的软件的配置文件
<code>/etc/X11</code>	X Window 系统 11 版本的的配置文件
<code>/etc/xml</code>	配置文件，比如 <code>catalogs</code> ，用于那些处理 XML (译者注：可扩展标记语言) 的软件的配置文件
<code>/home</code>	用户的主目录，包括保存的文件，个人配置，等等
<code>/lib</code>	<code>/bin/</code> 和 <code>/sbin/</code> 中的二进制文件的必需的库文件
<code>/lib<架构位数></code>	备用格式的必要的库文件。这样的目录是可选的，但如果它们存在的话肯定是有需要用到它们的程序
<code>/media</code>	可移动的多媒体 (如 CD-ROMs) 的挂载点。(出现于 FHS-2.3)
<code>/mnt</code>	临时挂载的文件系统
<code>/opt</code>	可选的应用程序软件包
<code>/proc</code>	以文件形式提供进程以及内核信息的虚拟文件系统，在 Linux 中，对应进程文件系统 (procfs) 的挂载点
<code>/root</code>	根用户的主目录
<code>/sbin</code>	必要的系统级二进制文件，比如， <code>init</code> , <code>ip</code> , <code>mount</code>
<code>/srv</code>	系统提供的站点特定数据
<code>/tmp</code>	临时文件 (另见 <code>/var/tmp</code>)。通常在系统重启后删除
<code>/usr</code>	二级层级存储用户的只读数据； 包含 (多) 用户主要的公共文件以及应用程序

目录	描述
<code>/usr/bin</code>	非必要的命令二进制文件（在单用户模式中不需要用到的）；用于所有用户
<code>/usr/include</code>	标准的包含文件
<code>/usr/lib</code>	库文件，用于 <code>/usr/bin/</code> 和 <code>/usr/sbin/</code> 中的二进制文件
<code>/usr/lib<架构位数></code>	备用格式库（可选的）
<code>/usr/local</code>	三级层次 用于本地数据，具体到该主机上的。通常会有下一个子目录， 比如 ， <code>bin/</code> ， <code>lib/</code> ， <code>share/</code> .
<code>/usr/local/sbin</code>	非必要系统的二进制文件，比如用于不同网络服务的守护进程
<code>/usr/share</code>	架构无关的（共享）数据.
<code>/usr/src</code>	源代码，比如内核源文件以及与它相关的头文件
<code>/usr/X11R6</code>	X Window 系统，版本号:11，发行版本：6
<code>/var</code>	各式各样的（Variable）文件，一些随着系统常规操作而持续改变的文件就放在这里，比如日志文件，脱机文件，还有临时的电子邮件文件
<code>/var/cache</code>	应用程序缓存数据。这些数据是由耗时的 I/O(输入/输出)的或者是运算本地生成的结果。这些应用程序是可以重新生成或者恢复数据的。当没有数据丢失的时候，可以删除缓存文件
<code>/var/lib</code>	状态信息。这些信息随着程序的运行而不停地改变，比如，数据库，软件包系统的元数据等等
<code>/var/lock</code>	锁文件。这些文件用于跟踪正在使用的资源
<code>/var/log</code>	日志文件。包含各种日志。
<code>/var/mail</code>	内含用户邮箱的相关文件
<code>/var/opt</code>	来自附加包的各种数据都会存储在 <code>/var/opt/</code> .
<code>/var/run</code>	存放当前系统上次启动以来的相关信息，例如当前登入的用户以及当前运行的 daemons(守护进程) .
<code>/var/spool</code>	该 spool 主要用于存放将要被处理的任务，比如打印队列以及邮件外发队列
<code>/var/mail</code>	过时的位置，用于放置用户邮箱文件
<code>/var/tmp</code>	存放重启后保留的临时文件

Linux 的文件类型

大多数文件仅仅是普通文件，他们被称为 **regular** 文件；他们包含普通数据，比如，文本、可执行文件、或者程序、程序的输入或输出等等

虽然你可以认为“在 Linux 中，一切你看到的皆为文件”这个观点相当保险，但这里仍有着一些例外。

- **目录**: 由其他文件组成的文件
- **特殊文件**: 用于输入和输出的途径。大多数特殊文件都储存在 **/dev** 中，我们将会在后面讨论这个问题。
- **链接文件**: 让文件或者目录出现在系统文件树结构上多个地方的机制。我们将详细地讨论这个链接文件。
- **(域)套接字**: 特殊的文件类型，和 TCP/IP 协议中的套接字有点像，提供进程间网络通讯，并受文件系统的访问控制机制保护。
- **命名管道**：或多或少有点像 sockets(套接字)，提供一个进程间的通信机制，而不用网络套接字协议。

现实中的文件系统

对于大多数用户和常规系统管理任务而言，“文件和目录是一个有序类树结构”是可以接受的。然而，对于电脑而言，它是不会理解什么是树，或者什么是树结构。

每个分区都有它自己的文件系统。想象一下，如果把那些文件系统想成一个整体，我们可以构思一个关于整个系统的树结构，不过这并没有这么简单。在文件系统中，一个文件代表着一个 **inode**(索引节点)，这是一种包含着构建文件的实际数据信息的序列号：这些数据表示文件是属于谁的，还有它在硬盘中的位置。

每个分区都有一套属于他们自己的 inode，在一个系统的不同分区中，可以存在有相同 inode 的文件。

每个 inode 都表示着一种在硬盘上的数据结构，保存着文件的属性，包括文件数据的物理地址。当硬盘被格式化并用来存储数据时(通常发生在初始系统安装过程，或者是在一个已经存在的系统中添加额外的硬盘)，每个分区都会创建固定数量的 inode。这个值表示这个分区能够同时存储各类文件的最大数量。我们通常用一个 inode 去映射 2-8k 的数据块。当一个新的文件生成后，它就会获得一个空闲的 inode。在这个 inode 里面存储着以下信息：

- 文件属主和组属主
- 文件类型(常规文件, 目录文件.....)
- 文件权限
- 创建、最近一次读文件和修改文件的时间
- inode 里该信息被修改的时间
- 文件的链接数(详见下一章)
- 文件大小
- 文件数据的实际地址

唯一不在 inode 的信息是文件名和目录。它们存储在特殊的目录文件。通过比较文件名和 inode 的数目, 系统能够构造出一个便于用户理解的树结构。用户可以通过 `ls -li` 查看 inode 的数目。在硬盘上, inodes 有他们独立的空间。